

Lab02_Dias

Isabella Ferreira Dias 266903

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.2
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(readr)
```

Laboratório 2: Processamento de Bases de Dados em Lote

Tarefa (retirada dos últimos slides da última aula)

Lendo 100.000 observações por vez (se você acredita que seu computador não tem memória suficiente, utilize 10.000 observações por vez), determine o percentual de vôos por Cia. Aérea que apresentou atraso na chegada (ARRIVAL_DELAY) superior a 10 minutos. As companhias a serem utilizadas são: AA, DL, UA e US. A estatística de interesse deve ser calculada para cada um dos dias de 2015. Para a determinação deste percentual de atrasos, apenas verbos do pacote dplyr e comandos de importação do pacote readr podem ser utilizados. Os resultados para cada Cia. Aérea devem ser apresentados em um formato de calendário.

Observação: a atividade descrita no slide pede para ler apenas 100 registros por vez. Aqui, mudamos para 100 mil registros por vez, para que haja melhor aproveitamento do tempo em classe.

1. Quais são as estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada (`ARRIVAL_DELAY > 10`)?

Para calcular o percentual de voos com atraso na chegada superior a 10 minutos para cada companhia aérea e para cada dia, não é necessário armazenar todas as observações da base. As estatísticas suficientes são:

- **Total de voos** da companhia aérea em cada dia (`total_voos`);
- **Número de voos com atraso superior a 10 minutos** (`voos_atrasados`).

Com essas duas estatísticas é possível calcular o percentual de atrasos por meio da expressão:

Perc = voos_atrasados / total_voos

Assim, apenas essas contagens precisam ser mantidas em memória para cada lote importado.

2. Crie uma função chamada `getStats` que, para um conjunto de qualquer tamanho de dados provenientes de `flights.csv.zip`, execute as seguintes tarefas (usando apenas verbos do `dplyr`):
 - (a) Filtre o conjunto de dados de forma que contenha apenas observações das seguintes Cias. Aéreas: AA, DL, UA e US;
 - (b) Remova observações que tenham valores faltantes em campos de interesse;
 - (c) Agrupe o conjunto de dados resultante de acordo com: dia, mês e cia. aérea;
 - (d) Para cada grupo em b., determine as estatísticas suficientes apontadas no item 1. e os retorne como um objeto da classe `tibble`;
 - (e) A função deve receber apenas dois argumentos:
 - `input`: o conjunto de dados (referente ao lote em questão);
 - `pos`: argumento de posicionamento de ponteiro dentro da base de dados. Apesar de existir na função, este argumento não será empregado internamente. É importante observar que, sem a definição deste argumento, será impossível fazer a leitura por partes.

```
getStats <- function(input, pos){  
  
  input %>%  
    filter(AIRLINE %in% c("AA", "DL", "UA", "US")) %>%  
  
    drop_na(AIRLINE, MONTH, DAY, ARRIVAL_DELAY) %>%  
  
    group_by(MONTH, DAY, AIRLINE) %>%  
  
    summarise(  
      total_voos = n(),  
      voos_atrasados = sum(ARRIVAL_DELAY > 10),  
    )  
}
```

```

    .groups = "drop"
  )
}

```

3. Utilize alguma função `readr::read_***_chunked` para importar o arquivo `flights.csv.zip`.

3.a. Configure o tamanho do lote (chunk) para 100 mil registros;

```

stats_chunks <- tibble()

chunk_size <- 100000

```

3.b Configure a função de callback para instanciar DataFrames aplicando a função `getStats` criada acima;

```

callback <- DataFrameCallback$new(function(x, pos){

  stats_chunks <-
    bind_rows(
      stats_chunks,
      getStats(x, pos)
    )

})

```

3.c. Configure o argumento `col_types` de forma que ele leia, diretamente do arquivo, apenas as colunas de interesse (veja nota de aula para identificar como realizar esta tarefa);

```

colunas <- cols_only(
  MONTH = col_integer(),
  DAY = col_integer(),
  AIRLINE = col_character(),
  ARRIVAL_DELAY = col_double()
)

read_csv_chunked(
  "flights.csv",
  callback = callback,
  chunk_size = chunk_size,
  col_types = colunas
)

```

```
# A tibble: 47,949 x 5
  MONTH DAY AIRLINE total_voos voos_atrasados
  <int> <int> <chr>      <int>      <int>
1     1     1     1 AA         1379         514
2     1     1     1 DL         1558         132
3     1     1     1 UA         1269         367
4     1     1     1 US         1028         201
5     1     2     2 AA         1508         649
6     1     2     2 DL         2261         406
7     1     2     2 UA         1411         493
8     1     2     2 US         1174         229
9     1     3     3 AA         1425         767
10    1     3     3 DL         2021         702
# i 47,939 more rows
```

4. Crie uma função chamada `computeStats` que:

4.a Combine as estatísticas suficientes para compor a métrica final de interesse (percentual de atraso por dia/mês/cia aérea);

```
computeStats <- function(stats_chunks){

  stats_chunks %>%
    group_by(MONTH, DAY, AIRLINE) %>%
    summarise(
      total_voos = sum(total_voos),
      voos_atrasados = sum(voos_atrasados),
      .groups = "drop"
    )

}
```

Nesta etapa, as estatísticas suficientes produzidas em cada lote são combinadas. Como um mesmo dia pode aparecer em diferentes blocos de leitura, é necessário somar o número total de voos e o número de voos atrasados para cada combinação de mês, dia e companhia aérea.

4.b Retorne as informações em um tibble contendo apenas as seguintes colunas: - Cia: sigla da companhia aérea; - Data: data, no formato AAAA-MM-DD (dica: utilize o comando `as.Date`); - Perc: percentual de atraso para aquela cia. aérea e data, apresentado como um número real no intervalo $[0,1]$.

```
stats <- computeStats(stats_chunks) %>%
  mutate(
    Cia = AIRLINE,
    Data = as.Date(paste(2015, MONTH, DAY, sep = "-")),
    Perc = voos_atrasados / total_voos
  ) %>%
  select(Cia, Data, Perc)

head(stats)
```

```
# A tibble: 6 x 3
  Cia   Data      Perc
  <chr> <date>    <dbl>
1 AA    2015-01-01 0.373
2 DL    2015-01-01 0.0847
3 UA    2015-01-01 0.289
4 US    2015-01-01 0.196
5 AA    2015-01-02 0.430
6 DL    2015-01-02 0.180
```

A tabela final é construída criando a coluna Data no formato AAAA-MM-DD e calculando o percentual de atrasos (Perc) como a razão entre o número de voos atrasados e o número total de voos daquele dia para cada companhia aérea. Por fim, são mantidas apenas as colunas solicitadas no enunciado: Cia, Data e Perc.

5. Produza um mapa de calor em formato de calendário para cada Cia. Aérea.

5.a. Instale e carregue os pacotes ggcal e ggplot2.

```
library(ggplot2)
library(ggcal)
```

5.b. Defina uma paleta de cores em modo gradiente. Utilize o comando `scale_fill_gradient`. A cor inicial da paleta deve ser #4575b4 e a cor final, #d73027. A paleta deve ser armazenada no objeto pal.

```
pal <- scale_fill_gradient(
  low = "darkblue",
  high = "pink"
)
```

5.c. Crie uma função chamada `baseCalendario` que recebe 2 argumentos a seguir: `stats` (tibble com resultados calculados na questão 4) e `cia` (sigla da Cia. Aérea de interesse). A função deverá: - Criar um subconjunto de `stats` de forma a conter informações de atraso e data apenas da Cia. Aérea dada por `cia`. - Para o subconjunto acima, montar a base do calendário, utilizando `ggcal(x, y)`. Nesta notação, `x` representa as datas de interesse e `y`, os percentuais de atraso para as datas descritas em `x`. - Retornar para o usuário a base do calendário criada acima.

```
baseCalendario <- function(stats, cia){  
  
  calendario <- stats %>%  
    filter(Cia == cia)  
  
  ggcal(  
    dates = calendario$Data,  
    fills = calendario$Perc  
  )  
  
}
```

A função `baseCalendario()` filtra a tabela de estatísticas para uma companhia aérea específica e cria a base do calendário utilizando a função `ggcal()`, relacionando cada data ao percentual de voos atrasados naquele dia.

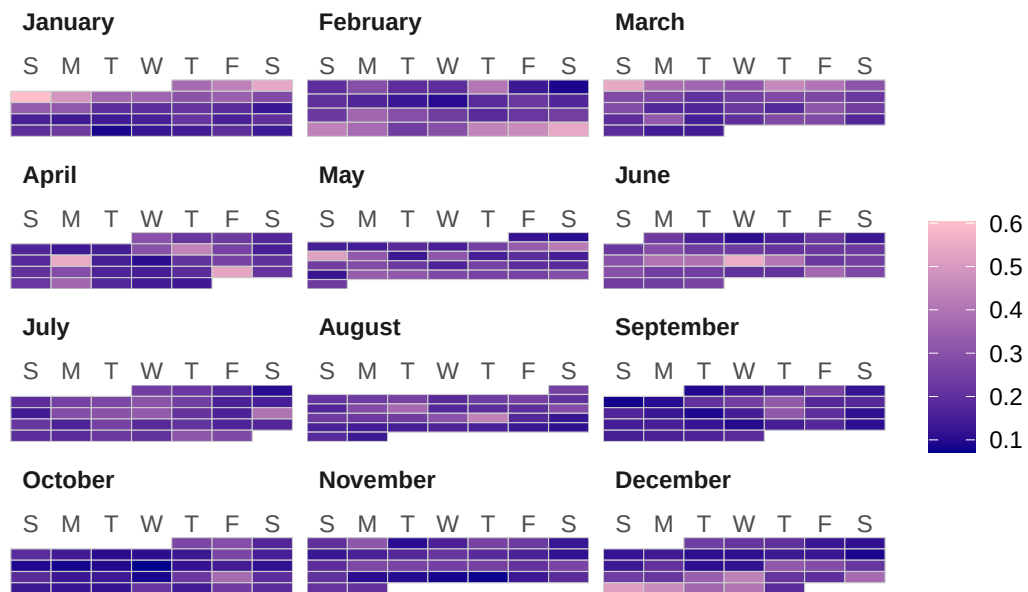
5.d. Executar a função `baseCalendario` para cada uma das Cias. Aéreas e armazenar os resultados, respectivamente, nas variáveis: `cAA`, `cDL`, `cUA` e `cUS`.

```
cAA <- baseCalendario(stats, "AA")  
cDL <- baseCalendario(stats, "DL")  
cUA <- baseCalendario(stats, "UA")  
cUS <- baseCalendario(stats, "US")
```

5.e. Para cada uma das Cias. Aéreas, apresente o mapa de calor respectivo utilizando a combinação de camadas do `ggplot2`. Lembre-se de adicionar um título utilizando o comando `ggtitle`. Por exemplo, `cXX + pal + ggtitle("Título")`.

```
cAA + pal + ggtitle("Percentual diário de atrasos - AA")
```

Percentual diário de atrasos – AA



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-24 19:42:14 -03"
```